



ITESO, Universidad Jesuita de Guadalajara

Entrega final Proyecto
IoT para Linux Embebido

Rodrigo Ramos Romero IE715082
Daniel Jiram Rodriguez Padilla IE703331
17/11/2025

Zamora Davalos Ricardo

Proyecto: MQTT con reconocimiento de comandos por voz

Tabla de Contenidos

Definición de la entrega	3
Aplicación	3
Desarrollo.....	4
Resultados	5

Definición de la entrega

La entrega final fue por mucho la entrega más difícil del proyecto. Se tuvieron que resolver bastantes problemas que no estaban planeados ni esperados. Esta entrega agrega el reconocimiento de los comandos que ya se pueden grabar con presionar el botón “Prev”.

Entrega 1

- Utilizar Mosquitto para la creación del Broker de MQTT a través del cual se comunicará la aplicación y todos los dispositivos.
- Probar este bróker con comandos desde otra CMD en el host de Windows.
- Crear la aplicación dentro de “MQTT Panel” en el celular y probar la comunicación con este bróker de MQTT.
- Lograr la conexión con el celular e interactuar con este utilizando comandos de Publish y Subscribe.

Entrega 2

- Agregar implementación del micrófono.
- Botón “Play” para grabar un comando de voz en formato .wav “Comando.wav”. graba por 2 segundos.
- Botón “Next” para escuchar el “Command.wav” y verificar que se haya grabado lo que queremos.
- Todo esto encima de la aplicación MQTT de la entrega 1

Entrega final

- Reconocimiento y distinción de cada comando
- Publish de MQTT correspondiente al comando

Aplicación

La aplicación funciona de esta manera:

1. Manda el paquete de conexión creado con apuntadores.
2. Espera el acknowledge, que el bróker haya permitido establecer una conexión.
3. Hace las subscripciones a los tópicos seleccionados.
4. Entra al loop de main, donde espera mensajes de MQTT en los tópicos suscritos, o un input del usuario para poder mandar un mensaje propio de Publish.
5. Escucha a los botones mientras está en el loop, graba o reproduce el comando.
6. Se graba un comando con “Play”.
7. Se escucha que se haya grabado lo que se desea con “Next”.
8. Se activa el reconocimiento con “Prev”, el cual manda el publish de MQTT correspondiente.

Desarrollo

Los problemas que surgieron durante el desarrollo fueron los siguientes:

- Calidad del audio

Primero que nada, se tiene que grabar todo con 48Khz. Sin embargo, la Pico no puede procesar tanto. Se tubo que hacer Downsampling del sonido a 16Khz para poder procesar bien todo. Esto hace mucho peor el reconocimiento.

- Python en PC

Se tuvo que usar específicamente la versión 3.9.0 de Python en una emulación. Se corrieron estos comandos en consola

```
PS C:\Keyword_spotting_model> py -3.9.0 -m venv venv
PS C:\Keyword_spotting_model> venv\Scripts\activate
(venv) PS C:\Keyword_spotting_model> python train_keyword_model_sklearn.py
```

- Python en Pico

La Pico ya tenía una versión de Python instalada. Pero resulta que tienen que ser exactamente las mismas versiones de Python con la que se entrena el modelo, y la que se usa el modelo. Se cambio el enfoque para usar código embebido en C.

- Comandos de voz eran demasiado parecidos

Todos los publish daban Light Off, estaba interpretando todo como “Off”. Se grabaron 20 grabaciones para educar al modelo con cada comando, ON, OFF, OPEN y CLOSE. Pero se cambió de usar comandos de voz a:

- Open: revolver cartas
- Close: golpecitos a la mesa.
- On: chasquidos
- Off: Chiflar

Con estos se obtuvo el siguiente análisis:

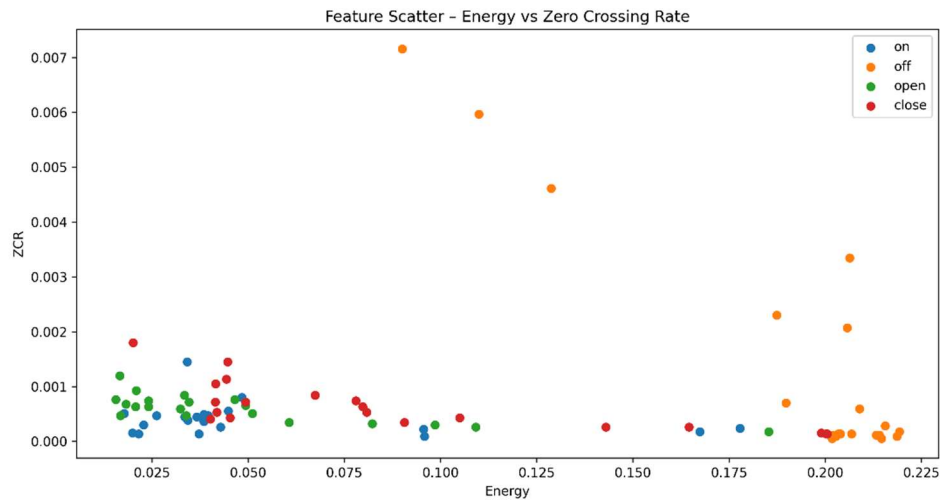


Figura 1: Análisis de los sonidos

Podemos ver que realmente solo Off es muy diferente a los demás. Lo que ciertamente lleva a resultados erróneos por el reconocimiento de comandos, pero mucho mejores que los que se obtenían con la voz.

Resultados

Las pruebas se realizaron primero probando el bróker creado de MQTT con Mosquitto, usando pings, subscribes y publishes desde consola en esta primera foto vemos el servidor funcionando perfectamente y como se necesita para la aplicación.

```

Command Prompt - mosquitto x
759788628: Received DISCONNECT from auto-9288CD29-BF19-CC08-4478-40086F39CD2F
759788628: Client auto-9288CD29-BF19-CC08-4478-40086F39CD2F disconnected.
759788631: Received PINGREQ from LinuxMQTTClient
759788631: Sending PINGRESP to LinuxMQTTClient
759788634: New connection from 192.168.0.246:51810 on port 1883.
759788634: New client connected from 192.168.0.246:51810 as auto-41D301C8-9EBC-4AAF-1D43-2983B7B57A31 (p2, c1, k60).
759788634: No will message specified.
759788634: Sending CONNACK to auto-41D301C8-9EBC-4AAF-1D43-2983B7B57A31 (0, 0)
759788634: Received PUBLISH from auto-41D301C8-9EBC-4AAF-1D43-2983B7B57A31 (d0, q0, r0, m0, 'Lights', ... (2 bytes))
759788634: Sending PUBLISH to LinuxMQTTClient (d0, q0, r0, m0, 'Lights', ... (2 bytes))
759788634: Sending PUBLISH to IoT (d0, q0, r0, m0, 'Lights', ... (2 bytes))
759788634: Received DISCONNECT from auto-41D301C8-9EBC-4AAF-1D43-2983B7B57A31
759788634: Client auto-41D301C8-9EBC-4AAF-1D43-2983B7B57A31 disconnected.
759788659: Received DISCONNECT from IoT
759788659: Client IoT disconnected.
759788659: New connection from 192.168.0.84:59152 on port 1883.
759788659: New client connected from 192.168.0.84:59152 as IoT (p2, c1, k300).
759788659: No will message specified.
759788659: Sending CONNACK to IoT (0, 0)
759788659: Received SUBSCRIBE from IoT
759788659: Lights (QoS 0)
759788659: IoT 0 Lights
759788659: Sending SUBACK to IoT
759788659: Received SUBSCRIBE from IoT
759788659: Door (QoS 0)
759788659: IoT 0 Door
759788659: Sending SUBACK to IoT
759788662: Received PINGREQ from LinuxMQTTClient
759788662: Sending PINGRESP to LinuxMQTTClient

```

Figura 2: MQTT Bróker corriendo en Windows CMD

Teniendo un bróker de MQTT funcional, el siguiente paso es preparar la interfaz. La meta del proyecto es lograr una interfaz completa con el celular, imprimiendo información de estados en este, y permitiendo actualizar estados y salidas del sistema total desde el celular. Se utilizó la aplicación “MQTT Panel” y esta es la interfaz creada:

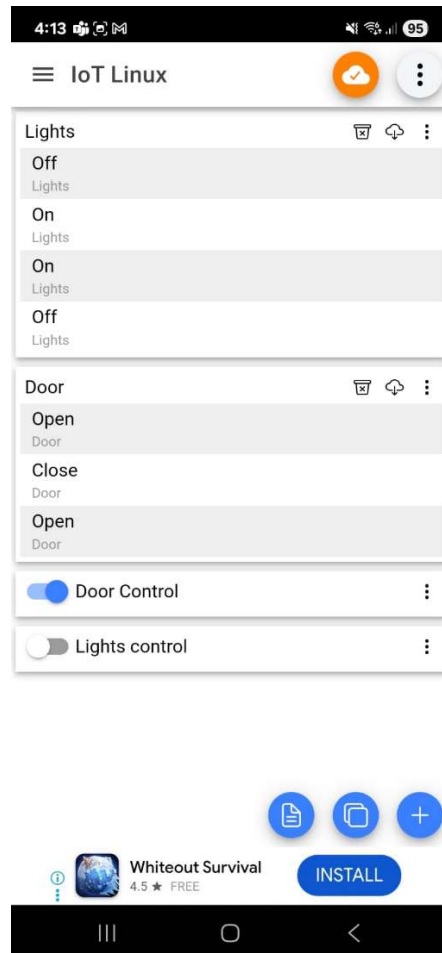


Figura 3: Aplicación de apoyo en el celular, MQTT Panel

Aquí podemos ver ya la aplicación funcionando. Imprimimos los últimos 3 estados o cambios de los tópicos “Door” y “Lights”. Los switches nos permiten actualizar manualmente el estado.

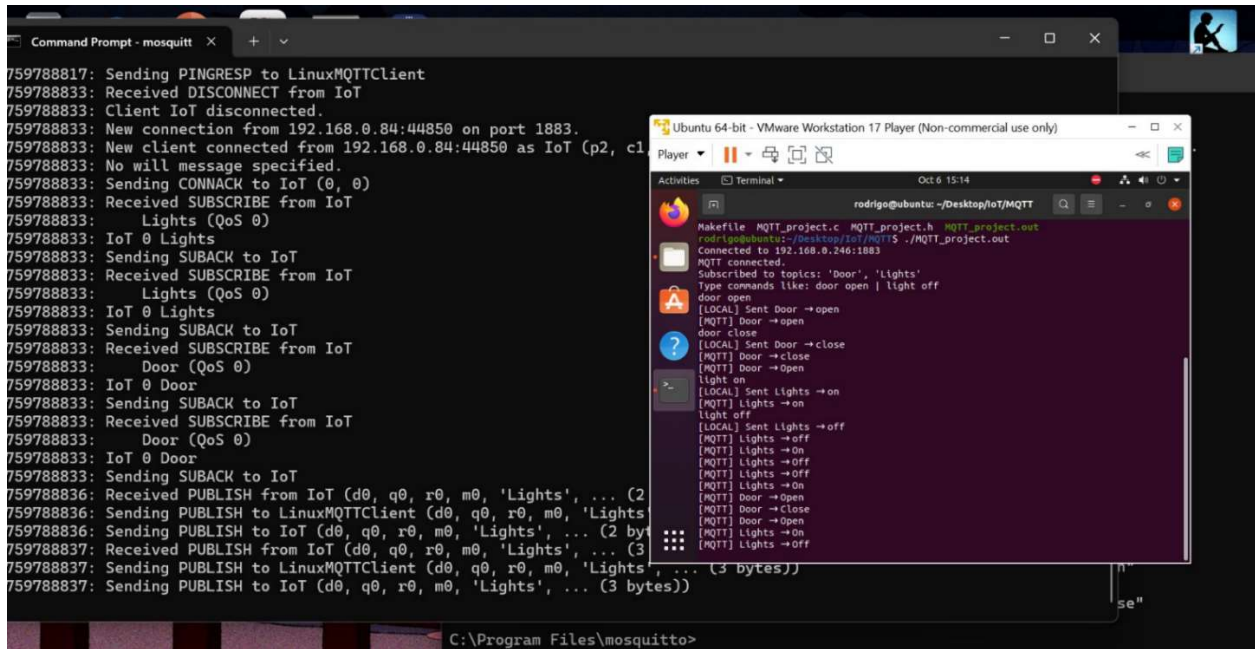


Figura 4: Aplicación funcionando en Consola

La aplicación funciona como fue esperado y definido en las metas. Se puede actualizar en la misma aplicación el estado de los dos tópicos o funcionalidades.

En esta foto podemos ver que el reconocimiento de voz en su versión original funcionaba muy bien en PC. Podemos ver que se reconocen bien los 4 comandos a pesar de su precisión de 81.25%

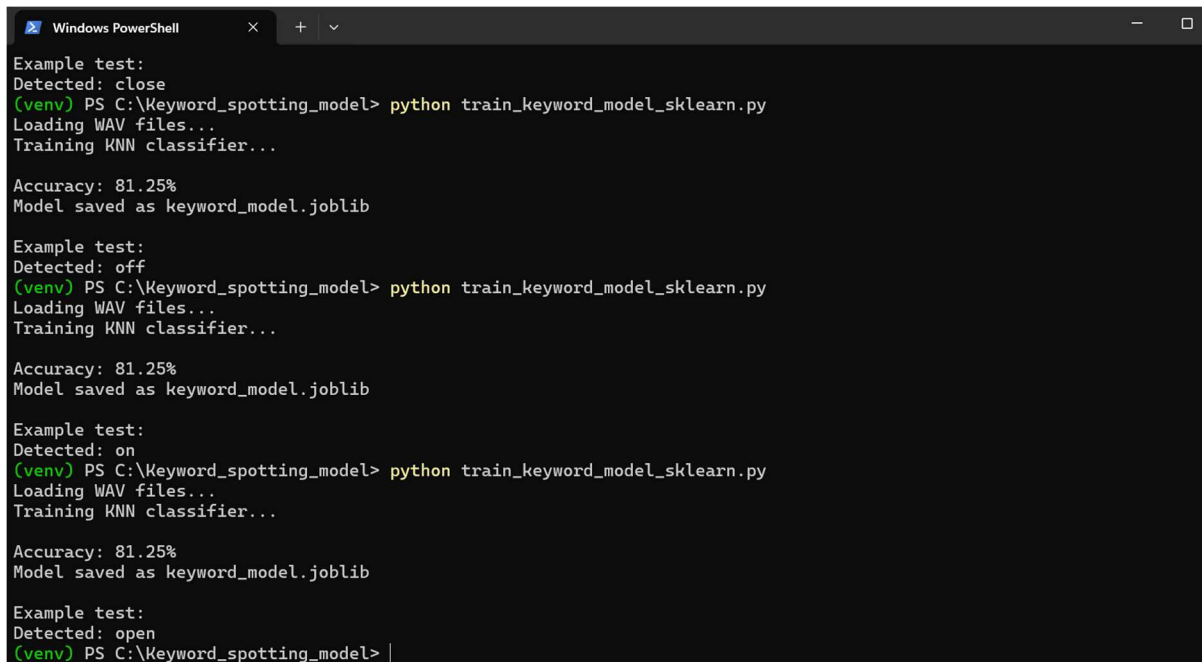
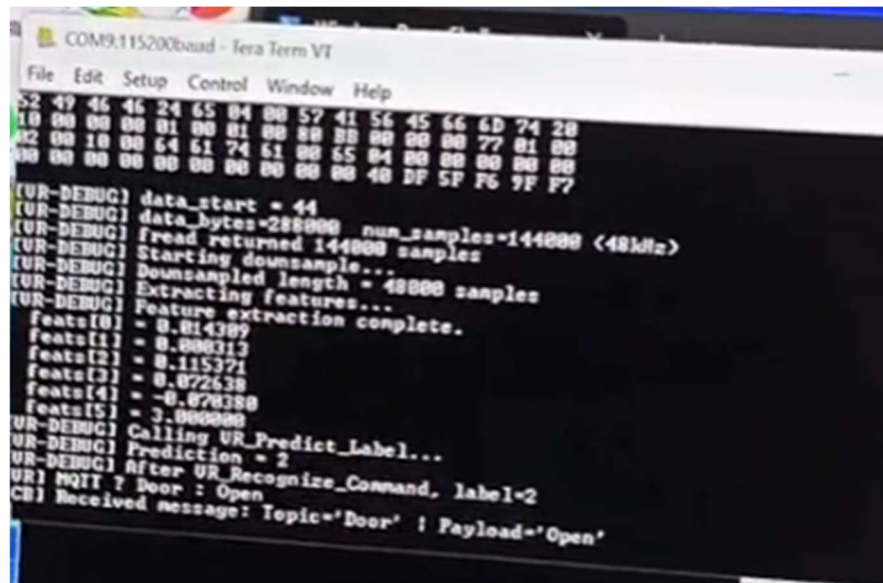


Figura 5: Reconocimiento de comandos en PC

En la Pico “funciona” el modelo. No se lograron que los resultados fueran 100% precisos, pero si se pueden obtener los diferentes publish.



```
COM9:115200baud - Tera Term VI
File Edit Setup Control Window Help
52 49 46 46 24 65 04 00 57 41 56 45 66 6D 74 20
10 00 00 00 01 00 01 00 00 00 00 00 00 00 00 00
02 00 10 00 64 61 74 61 00 55 04 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[UR-DEBUG] data_start = 44
[UR-DEBUG] data_bytes=288000 num_samples=144000 (48kHz)
[UR-DEBUG] fread returned 144000 samples
[UR-DEBUG] Starting downsample...
[UR-DEBUG] Downsampled length = 48000 samples
[UR-DEBUG] Extracting features...
[UR-DEBUG] Feature extraction complete.
feats[0] = 0.814389
feats[1] = 0.800313
feats[2] = 0.115371
feats[3] = 0.872638
feats[4] = -0.878388
feats[5] = 3.000000
[UR-DEBUG] Calling UR_Predict_Label...
[UR-DEBUG] Prediction = 2
[UR-DEBUG] After UR_Recognize_Command, label=2
UR! MQTT ? Door : Open
CB! Received message: Topic='Door' ! Payload='Open'
```

Figura 6: Resultados de la Pico